

CS308 Final Project Defense Script and QA

Group 8

Topic: Open-Vocabulary Object Detection and Visual Grounding

Slides: [presentation/cv_final_project_presentation.pdf](#)

10-15 Minute Defense Script

Slide 1. Title

各位老师同学好，我们是第八组。我们的项目题目是 Open-Vocabulary Object Detection and Visual Grounding。

这次项目我们主要做的不是提出一个新的检测模型，而是围绕课程第四个选题，把三个代表性的开放词表检测和视觉 grounding 模型放在同一个可复现评测框架下进行比较。我们最终比较了 OWL-ViT Base、Grounding DINO Tiny 和 YOLO-World v2 Small，并在 COCO 和 RefCOCO 上分别评估检测能力和 grounding 能力。

我希望这次汇报最后能回答一个很实际的问题：如果我们现在要在项目或者应用里选择一个开放词表检测模型，到底应该选择更准的 Grounding DINO，还是更快的 YOLO-World，或者更简单轻量的 OWL-ViT。

Slide 2. Roadmap

汇报会分成五部分。第一是任务背景和挑战，第二是相关工作和模型选择，第三是我们实现的统一评测 pipeline，第四是 COCO、RefCOCO 和 ablation 的实验结果，最后总结 limitation 和 takeaways。

这里先强调一点：我们报告和 slides 里面的 headline numbers 都来自 full-split 实验。之前仓库里保留的 100-row 或 500-image 结果主要是 debug 和 sanity check，不作为正式结论。

Slide 3. What Problem Are We Solving?

传统 object detection 通常是在固定类别集合上训练和预测，比如 COCO 的 80 类。如果我们想检测一个训练时没有作为固定类别出现的新概念，往往需要重新标注和训练。

Open-vocabulary detection 的目标是让模型直接用文本 prompt 来指定要检测什么。例如输入 "remote control"、"red car" 或者更开放的物体名称，模型应该返回对应区域。

Visual grounding 更进一步。它不是只问图里有没有某一类物体，而是给一句 referring expression，比如 "the man on the left" 或 "the bottle behind the laptop"，要求模型定位这一句话描述的那个具体 instance。所以 grounding 不仅需要识别类别，还需要处理颜色、位置、关系和多个相似物体之间的歧义。

因此我们的目标是：复现三个代表性模型，搭建统一的评测与可视化代码，并严格比较 accuracy、speed、memory 和失败模式。

Slide 4. Related Work and Model Choice

我们选择三个模型，是因为它们代表了不同设计点。

OWL-ViT 是一个比较直接的 vision-language transfer baseline。它把文本 query 和图像 token 放到相似度空间中做 detection，优点是接口简单、显存低，适合作为 compact baseline。

Grounding DINO 是 grounding-oriented 的 transformer detector。它有更强的跨模态融合机制，在 open-vocabulary detection 和 phrase grounding 上通常更准。我们把它作为 accuracy-first baseline。

YOLO-World 是实时开放词表检测模型。它把 YOLO 风格的高吞吐检测和文本类别嵌入结合起来，所以非常适合部署导向的比较。

选择这三个模型的核心原因是：它们覆盖了 accuracy、speed、memory 三个维度的不同 trade-off，这也和课程项目强调的验证和分析过程比较匹配。

Slide 5. What We Built

这一页总结我们实际完成的工作量。

首先，任务覆盖上，我们同时做了 open-vocabulary object detection 和 visual grounding，不只是 demo。Detection 部分用 COCO val2017 的 5,000 张图片和 80 个类别 prompt；Grounding 部分用 RefCOCO validation 的完整 split，并且对每个 region 的所有 expressions 都做评测。

其次，模型覆盖上，三个模型都接入了统一框架。指标方面，COCO 使用 AP、AP50、AP75、AR100；RefCOCO 使用 Acc@0.5、Acc@0.75 和 mean IoU；效率方面记录 pipeline FPS 和 peak VRAM。

另外我们做了 threshold sensitivity 和 OWL-ViT NMS diagnostic。这部分的意义是说明 post-processing 不只是可视化细节，它会真实影响 AP 和 recall。

最后是复现性。我们统一配置、缓存 manifest、给 RefCOCO 增加本地 parquet fallback，并且保留服务器上的 logs 和 metrics JSON。

Slide 6. Implementation Map

实现上，我们把每个模型的 API 差异封装在 model wrapper 里。

COCO 部分有对应的 `evaluate_coco_*.py` 脚本。它们负责准备图片、运行模型、保存 COCO 格式预测、记录 timing 和显存，然后调用统一 evaluator。

RefCOCO 部分由 `evaluate_refcoco.py` 完成。这里我们做了一个比较重要的补强：早期版本只取前 100 行 region，而且每个 region 只用第一句 expression，这更像 diagnostic subset。最终版本改成 full validation split，并且 expression mode 设置为 all，也就是 8,811 rows、25,080 expressions。

共享基础设施包括 `coco_eval_utils.py`、`generate_report_figures.py` 和服务器上的 `run_full_suite_4gpu_server.sh`。这些脚本保证所有模型最终写出类似结构的 metrics、logs 和 figures。

Slide 7. Unified Pipeline

这一页是整体 pipeline。

输入是 image 和 text prompt。对于 COCO，prompt 是 80 个类别名；对于 RefCOCO，prompt 是每条 referring expression。中间经过三个模型 wrapper，分别处理 OWL-ViT、Grounding DINO 和 YOLO-World 的不同输入格式。

然后我们把输出统一成 box、score、label 或 expression id、timing 和 memory。统一格式之后，后面的 evaluator 就可以对所有模型使用同一套逻辑。

这里还有两个工程细节。第一，服务器 4-GPU 脚本会把不同实验分配到不同 GPU，并写入 timestamped logs。第二，RefCOCO 使用本地 Hugging Face parquet snapshot 作为 fallback，因为服务器上 online rows API 曾经失败。这样正式 rerun 不会依赖临时网络状态。

这个 pipeline 的核心价值是公平比较：same data, same metrics, same reporting path。

Slide 8. Experimental Setup

正式实验是在 4 x TITAN RTX 服务器上跑的。环境是 NVIDIA driver 550.120、CUDA 12.4、PyTorch 2.6.0+cu124、Torchvision 0.21.0+cu124、Transformers 5.5.3 和 Ultralytics 8.4.70。

COCO 正式评测使用完整 val2017，共 5,000 images。RefCOCO 正式评测使用 full validation split，共 8,811 rows 和 25,080 expressions。

指标上，COCO 用标准 COCO AP 系列，RefCOCO 用 top-1 localization accuracy。也就是说每个 expression 只取模型最高分预测框，然后看它和 ground-truth box 的 IoU 是否超过 0.5 或 0.75。

这一页的重点是：小规模 subset 只用于调试，最终表格和报告中的结论都来自 full split。

Slide 9. COCO Detection Results

这里是 COCO detection 的结果。

Grounding DINO 是准确率最强的模型，full COCO AP 是 0.421，AP50 是 0.557，AP75 是 0.457，AR100 是 0.536。YOLO-World 的 AP 是 0.366，AP50 是 0.510，AP75 是 0.398，AR100 是 0.547，说明它召回能力很好。OWL-ViT 的 AP 是 0.237，AP50 是 0.385，AP75 是 0.249，AR100 是 0.469，是三个模型里最低的。

这个结果可以这样理解：Grounding DINO 的 confidence ranking 和 localization 质量更好，所以 AP 和 AP75 都最高；YOLO-World 能提出很多有用候选框，所以 recall 很强，但排序和精细定位稍弱；OWL-ViT 作为简单 baseline 在小物体上尤其吃亏。

所以如果目标是 COCO-style detection accuracy，Grounding DINO 是最佳选择，而 YOLO-World 更像是效率优先的选择。

Slide 10. Efficiency and Deployment Trade-off

但是准确率不是唯一指标。效率结果显示三者差异非常明显。

YOLO-World 达到 84.6 pipeline FPS，peak VRAM 大约 715 MB，是速度最强的模型。OWL-ViT 是 24.3 FPS，peak VRAM 350 MB，是显存最低的。Grounding DINO 只有 4.5 FPS，并且 peak VRAM 约 2357 MB。

因此 Grounding DINO 适合 offline annotation、dataset mining 或者 human-in-the-loop labeling 这种追求质量的场景。YOLO-World 更适合实时视频、交互式应用或机器人原型。OWL-ViT 虽然不够准，但低显存、接口简单，适合作为 baseline 和轻量参考。

一句话总结就是：accuracy winner is not throughput winner。

Slide 11. RefCOCO Grounding Results

这一页是我们最重要的补强之一：RefCOCO full-split grounding。

Grounding DINO 的 Acc@0.5 是 51.1%，Acc@0.75 是 45.6%，mean IoU 是 0.514，三个指标都最高。OWL-ViT 的 Acc@0.5 是 42.5%，Acc@0.75 是 34.2%，mean IoU 是 0.423。YOLO-World 的 Acc@0.5 是 41.4%，Acc@0.75 是 36.2%，mean IoU 是 0.422，但它在速度上非常快，RefCOCO pipeline FPS 达到 115.1。

RefCOCO 比 COCO 更难，因为它要求模型定位自然语言表达描述的具体 instance，而不是简单检测类别名。比如同一张图中有多个人或多辆车时，模型必须理解 "left"、"white"、"behind" 这样的属性或关系。

这个实验说明 Grounding DINO 的跨模态融合确实对 phrase-level grounding 更有帮助，也证明我们项目不是只做开放类别检测，还覆盖了视觉 grounding 的正式评测。

Slide 12. Threshold Sensitivity and NMS

这一页是 ablation。

不同模型的 score calibration 不一样，所以一个固定 threshold 在不同模型之间不能直接比较。我们做的是 within-model threshold sensitivity，也就是对同一个模型逐渐提高阈值，看 AP 和 recall 怎么变化。

结果很清楚：提高 threshold 通常会让可视化更干净，但会降低 AP 和 AR。比如 OWL-ViT 从 threshold 0.01 提高到 0.20，AP 从 0.237 降到 0.184，AR100 从 0.469 降到 0.255。Grounding DINO 从 0.20 到 0.40，AP 从 0.421 降到 0.362，AR100 从 0.536 降到 0.418。YOLO-World 从 0.001 到 0.25，AP 从 0.366 降到 0.322，AR100 从 0.547 降到 0.393。

OWL-ViT 的 NMS diagnostic 也有类似结论。NMS 能减少重复框，让图片看起来更清爽，但在 crowded scene 中会误删有效候选，导致 AP 和 AR 下降。我们在 100-image subset 上看到 AP 从 0.336 降到 0.323，AR100 从 0.520 降到 0.475。

所以我们的正式评测和展示图采用了分离策略：低阈值预测用于量化指标，更干净的设置只用于 qualitative visualization。

Slide 13. Qualitative Examples and Failure Modes

这里展示两组 qualitative examples：workspace 和 traffic，对比 OWL-ViT 与 Grounding DINO。

在 workspace 场景里，两者都能检测主要物体，但 Grounding DINO 的框通常更紧、更稳定。在 traffic 场景中，小物体、遮挡物体和相近类别更容易出错。

我们总结了三类失败模式。第一是 localization error，找到了正确物体但框太松或太紧。第二是 semantic error，把相关但错误的类别选出来。第三是 grounding error，类别对了，但没有选中 expression 指定的那个 instance。

第三类错误是 COCO 不容易暴露、但 RefCOCO 能暴露的，这也是为什么我们把 RefCOCO full evaluation 作为最终项目补强重点。

Slide 14. Limitations and Takeaways

我们也诚实说明 limitation。

第一，没有 fine-tuning，也没有训练新的 calibration 或 reranking 模块。这样做的好处是比较公平和可复现，但绝对性能可能不是每个模型的最优上限。

第二，COCO 和 RefCOCO 仍然是 curated benchmarks，不等于真实开放世界。未来可以扩展到 LVIS、ODinW 或 Flickr30K Entities。

第三，runtime 依赖硬件、precision、输入分辨率和软件版本，所以我们报告的是自己服务器环境下的可复现实测值。

最终 takeaway 是：Grounding DINO 在 accuracy 上最好；YOLO-World 在 speed 和 efficiency 上最好；OWL-ViT 是简单低显存 baseline。整体上，我们做的是一个统一、完整、可复现的 open-vocabulary detection 和 grounding benchmark。

Slide 15. Thank You

谢谢老师和同学。我们欢迎提问。

如果需要一句话概括我们的项目：我们不是只跑了一个 demo，而是把三个代表性开放词表检测模型放进统一评测框架，在 full COCO 和 full RefCOCO 上给出了可复现的准确率、速度、显存和失败模式分析。

Backup Slides

Slide 16. Exact Full-Split Numbers

如果老师问具体数值，可以直接回到 backup table。

COCO 上：

- OWL-ViT: AP 0.237, AP50 0.385, AR100 0.469, FPS 24.3
- Grounding DINO: AP 0.421, AP50 0.557, AR100 0.536, FPS 4.5
- YOLO-World: AP 0.366, AP50 0.510, AR100 0.547, FPS 84.6

RefCOCO 上：

- OWL-ViT: Acc@0.5 0.425, Acc@0.75 0.342, mean IoU 0.423, FPS 29.9
- Grounding DINO: Acc@0.5 0.511, Acc@0.75 0.456, mean IoU 0.514, FPS 3.1
- YOLO-World: Acc@0.5 0.414, Acc@0.75 0.362, mean IoU 0.422, FPS 115.1

Slide 17. Reproducibility Checklist

如果老师问怎么复现，可以回答：

正式服务器脚本是：

```
1 | bash scripts/run_full_suite_4gpu_server.sh
```

关键配置是：

```
1 | COCO_MAX_IMAGES=0
2 | REFCOCO_MAX_ROWS=0
3 | REFCOCO_EXPRESSION_MODE=all
```

这表示 COCO 和 RefCOCO 都跑 full split，RefCOCO 不只用第一句表达，而是使用 all expressions。最终 metrics 和 logs 在 `outputs/server_2026_06/` 和 `logs/server_2026_06/` 下。

1-2 Minute QA Preparation

Q1. 你们的方法有什么创新？是不是只是调用模型？

推荐回答：

我们没有声称提出新 detector。我们的贡献是 engineering reproduction 和 rigorous evaluation。具体包括三个方面：第一，把 OWL-ViT、Grounding DINO、YOLO-World 三种不同 API 的模型统一到一个输出 schema；第二，在 full COCO 和 full RefCOCO 上做可复现评测，而不是只展示 demo 或小 subset；第三，加入 runtime、VRAM、threshold sensitivity 和 NMS diagnostic，分析 accuracy 和

deployment trade-off。所以重点是系统性比较和可靠实验，而不是模型结构创新。

Q2. 为什么选择这三个模型？

推荐回答：

因为它们覆盖了三个典型设计点。OWL-ViT 是简单 vision-language transfer baseline，Grounding DINO 是 grounding accuracy 代表，YOLO-World 是 real-time open-vocabulary detector。放在一起可以比较准确率、速度和显存，而不是只比较同一类模型。

Q3. 为什么 Grounding DINO 比 YOLO-World 准，但更慢？

推荐回答：

Grounding DINO 使用更强的 transformer 和 cross-modal fusion，对文本和图像交互建模更充分，所以在 COCO AP 和 RefCOCO grounding 上更强。但这种结构计算开销更大。YOLO-World 继承 YOLO-style 检测框架，吞吐量更高，所以速度明显领先，但对复杂 referring expression 的 instance selection 没有 Grounding DINO 稳定。

Q4. RefCOCO 为什么重要？COCO 不够吗？

推荐回答：

COCO 主要评估 category-level detection，例如输入 "person" 或 "car" 后检测所有类别实例。RefCOCO 评估的是 phrase-level grounding，例如 "the man on the left"。这要求模型理解属性、位置和关系，并选择具体 instance。所以 RefCOCO 更贴近 visual grounding 任务，也是课程第四个选题中 grounding 部分的关键证据。

Q5. 为什么之前有 100-row 和 500-image 结果？会不会影响正式结论？

推荐回答：

不会。100-row 和 500-image 是为了调试模型、数据加载、prompt 格式和服务器环境。最终报告和 slides 的 headline results 都来自 full split: COCO val2017 5,000 images, RefCOCO val 25,080 expressions。我们在 README 和报告中也明确说明了 subset 结果只作为 diagnostic。

Q6. 为什么提高 threshold 后 AP 下降？不是去掉低质量框了吗？

推荐回答：

阈值提高确实会让可视化更干净，但 AP 是 precision-recall curve 上综合指标。如果 threshold 太高，很多正确但置信度不够高的候选框会被删掉，recall 下降，AP 也会下降。我们的实验说明 visualization setting 和 quantitative evaluation setting 应该分开处理。

Q7. 为什么 OWL-ViT 加 NMS 后反而更差？

推荐回答：

NMS 会删除重叠框，这对可视化有帮助。但在 crowded scene 里，多个相邻或重叠物体都可能是真实目标，或者一个正确候选被另一个相近候选压掉。我们在 OWL-ViT 100-image diagnostic 上看到 AP 和 AR 都下降，所以正式评测中没有额外使用 NMS。

Q8. 为什么不 fine-tune?

推荐回答:

课程题目强调 open-vocabulary detection 和 grounding, 我们希望比较 pretrained models 在 download-and-evaluate 场景下的真实表现。Fine-tuning 会引入额外训练数据、超参数和不公平因素。我们选择不 fine-tune, 是为了让三个模型在同一协议下直接比较。

Q9. 实验结果和论文 reported number 不一样怎么办?

推荐回答:

这很正常。我们的目标不是复现某篇论文的 exact setting, 而是在同一硬件、同一 prompt、同一数据 split、同一 evaluator 下横向比较三个模型。不同论文可能使用不同 checkpoint、resolution、prompt template、threshold、NMS 和 hardware, 所以绝对数值不能直接一一对应。

Q10. 如果要部署, 你们推荐哪个模型?

推荐回答:

如果任务追求最高检测或 grounding accuracy, 推荐 Grounding DINO。如果是实时视频或交互式应用, 推荐 YOLO-World, 因为它在我们服务器上达到 84.6 COCO FPS 和 115.1 RefCOCO FPS。OWL-ViT 更适合低显存 baseline 或教学参考。

Q11. 项目还有什么可以继续改进?

推荐回答:

可以从三方面扩展。第一, 用 LVIS 或 ODinW 测长尾开放类别。第二, 用 Flickr30K Entities 测 caption-style grounding。第三, 做 prompt paraphrase robustness, 比如比较 "bike"、"bicycle"、"person riding a bicycle" 对检测结果的影响。也可以进一步做 score calibration 或 learned reranking。

Q12. 你们如何保证复现性?

推荐回答:

我们保证了几件事: 统一 environment requirements; COCO 和 RefCOCO 都有 manifest/cache; RefCOCO 支持本地 parquet fallback; 服务器脚本统一配置并写入 timestamped logs; 所有 final metrics 都保存为 JSON; report figures 由脚本从 metrics 生成。这样可以从仓库重新跑并追溯每个数字。

Short Emergency Version

如果时间只剩 5 分钟, 可以按下面顺序讲:

1. 我们做的是 Project 4: open-vocabulary detection and visual grounding。
2. 比较三个 pretrained baselines: OWL-ViT、Grounding DINO、YOLO-World。
3. 贡献是统一评测框架、full COCO、full RefCOCO、speed/VRAM、threshold/NMS ablation。
4. COCO 结论: Grounding DINO AP 0.421 最准, YOLO-World AP 0.366 但 FPS 84.6 最快, OWL-ViT AP 0.237。
5. RefCOCO 结论: Grounding DINO Acc@0.5 51.1% 最强; YOLO-World 速度 115.1 FPS 最强。
6. Ablation 结论: 更高 threshold 和 NMS 可以让图更干净, 但会降低 recall/AP。

7. 总结: Grounding DINO 适合 accuracy-first, YOLO-World 适合 deployment, OWL-ViT 是简单低显存 baseline。